

# 动态上下文发现

Jan 6, 2026 由 Jediah Katz 在 研究

## 目录

用于动态上下文发现的文件

1. 将较长的工具响应转换为文件
  2. 在摘要过程中引用对话历史
  3. 支持 Agent Skills 开放标准
  4. 高效地仅加载所需的 MCP 工具
  5. 将所有集成终端会话视为文件
- 简单抽象



代码 Agent 正在迅速改变软件的开发方式。它们的快速进步既来自更强的 Agent 模型，也来自更好的上下文工程，用于引导其行为。

Cursor 的 agent harness（我们为模型提供的指令和工具）会针对我们支持的每一个新的前沿模型进行单独优化。不过，我们还可以在一些对所有模型都适用的上下文工程方面做改进，比如我们如何收集上下文，以及如何在长时间交互过程中优化 token 使用。

随着模型在充当 Agent 方面的能力不断提升，我们发现，预先提供更少的细节，反而能让 Agent 更容易自主地按需提取相关上下文。我们将这一模式称为 **动态上下文发现 (dynamic context discovery)**，与始终被包含的 **静态上下文 (static context)** 相对。

## 用于动态上下文发现的文件

动态上下文发现的 token 使用效率要高得多，因为只会将必要的数据引入上下文窗口。同时，它还能通过减少上下文窗口中可能造成混淆或互相矛盾的信息量来提升 agent 的回复质量。

以下是我们在 Cursor 中使用动态上下文发现的做法：

1. 将较长的工具响应转化为文件



登录

下载

4. 向本地集成终端添加所需的 MCP 工具

5. 将所有集成终端会话视为文件

## 1. 将较长的工具响应转换为文件

工具调用可能会因为返回体积巨大的 JSON 响应而显著增加上下文窗口的大小。

对于 Cursor 中的自研工具，比如编辑文件和搜索代码库，我们可以通过合理的工具定义和精简的响应格式来避免上下文膨胀，但第三方工具（如 shell 命令或 MCP 调用）并不能天然享受同样的优化。

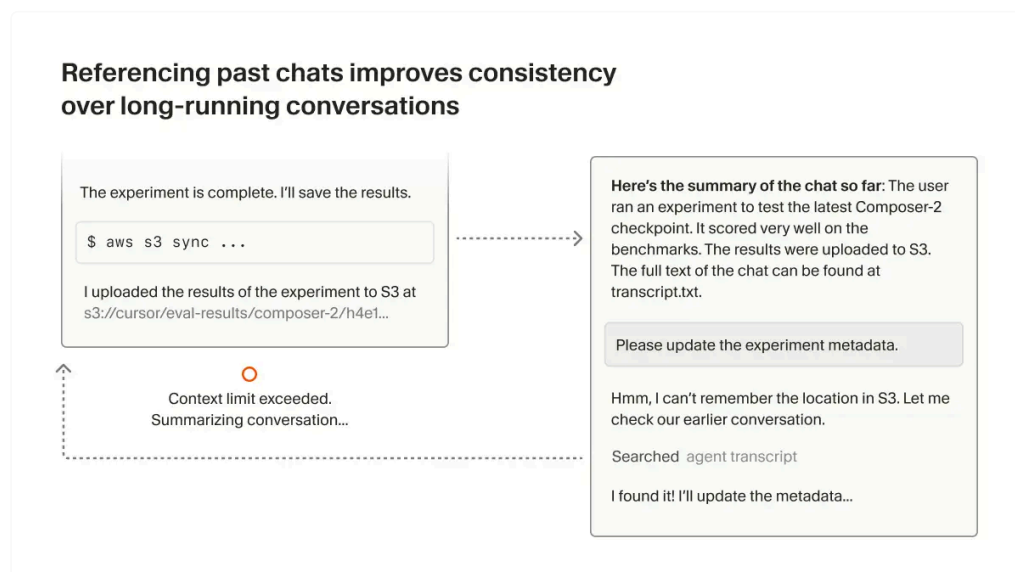
常见的做法是，代码 Agent 会截断较长的 shell 命令输出或 MCP 结果。这可能导致数据丢失，其中可能包括你希望保留在上下文中的重要信息。在 Cursor 中，我们则是把输出写入文件，并赋予 Agent 读取该文件的能力。Agent 会调用 `tail` 来检查末尾内容，如有需要再继续向后读取更多内容。

这样在接近上下文上限时，就能减少不必要的额外总结。

## 2. 在摘要过程中引用对话历史

当模型的上下文窗口被填满时，Cursor 会触发一次摘要步骤，为 Agent 提供一个全新的上下文窗口，其中包含它迄今为止工作的摘要。

但由于这是对上下文的有损压缩，Agent 的掌握情况在摘要之后可能会变差，可能会忘记任务中的关键细节。在 Cursor 中，我们将对话历史作为文件提供，以提升摘要的质量。





登录

下载

简单来说，它可以让你从上下文动态地找出感兴趣的。

### 3. 支持 Agent Skills 开放标准

Cursor 支持 Agent Skills，这是一种用于为编码 Agent 扩展专用能力的开放标准。与其他类型的 Rules 类似，Skills 由文件定义，这些文件会告诉 Agent 如何执行特定领域的任务。

Skills 还包括名称和描述，可以作为“静态上下文”包含在系统提示词中。随后，Agent 可以进行动态上下文发现，使用诸如 grep 和 Cursor 的语义搜索等工具自动引入相关的 Skills。

Skills 还可以打包与任务相关的可执行文件或脚本。由于它们本质上只是文件，Agent 可以轻松找到与某个特定 Skill 相关的内容。

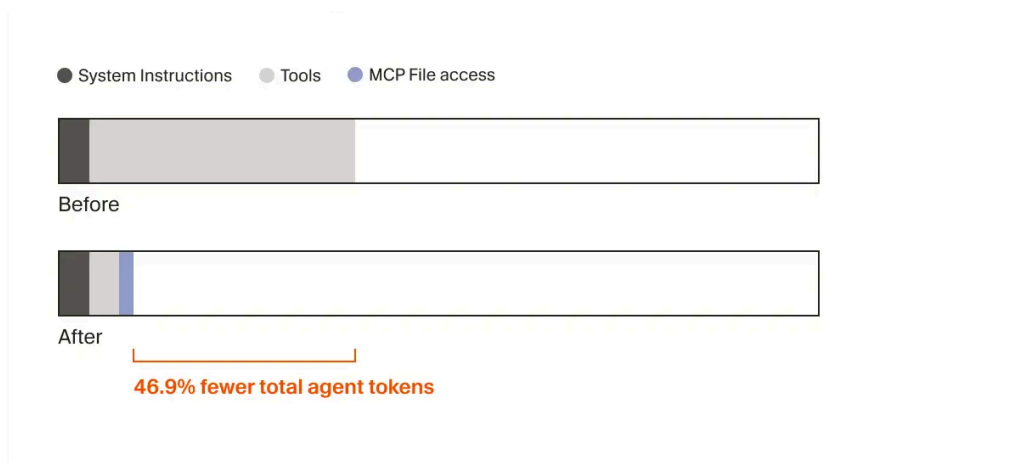
### 4. 高效地仅加载所需的 MCP 工具

MCP 有助于访问受 OAuth 保护的资源，比如生产环境日志、外部设计文件，或企业内部的上下文和文档。

有些 MCP 服务器包含很多工具，且往往带有很长的描述，这会显著膨胀上下文窗口。即使这些工具始终被包含在提示中，其中大部分实际上并不会被使用。如果你使用多个 MCP 服务器，这个问题会被进一步放大。

指望每个 MCP 服务器都为此进行优化并不现实。我们认为，降低上下文占用是编码 Agent 的责任。在 Cursor 中，我们通过将工具描述同步到一个文件夹，为 MCP 提供了动态上下文发现能力。<sup>1</sup>

Agent 现在只会收到一小段静态上下文（包括工具名称），并在任务需要时再去查找具体工具。在一次 A/B 测试中，我们发现：在会调用 MCP 工具的运行中，这一策略将 **Agent 的总 token 消耗减少了 46.9%**（这一结果在统计上显著，但会随已安装 MCP 的数量产生较大波动）。



这种基于文件的方案还带来了一个好处：可以向 Agent 传达 MCP 工具的状态。比如，以前如果某个 MCP 服务器需要重新认证，Agent 会完全“遗忘”这些工具，让用户摸不着头脑。现在，它可以主动提示用户进行重新认证。

## 5. 将所有集成终端会话视为文件

过去你需要把终端会话的输出复制粘贴到 Agent 的输入中，Cursor 现在会自动将集成终端的输出同步到本地文件系统。

这样你就可以轻松询问“为什么我的命令失败了？”，并让 Agent 理解你具体在引用什么。由于终端历史可能很长，Agent 可以只对相关输出进行 grep，这对于像服务器这类长时间运行的进程所产生的日志尤其有用。

这与基于 CLI 的编码 Agent 所看到的情况相似：它们同样可以在上下文中访问先前的 shell 输出，但这里是动态发现的，而不是静态注入的。

## 简单抽象

目前还不清楚，文件是否会成为基于 LLM 的工具的最终接口形式。

不过，随着编码 Agent 的快速发展，文件一直是一种简单而强大的原语，相比再造出一种无法充分兼顾未来的抽象层，也是更安全的选择。请持续关注，我们将在这个方向分享更多令人兴奋的进展。

这些改进会在未来几周内向所有用户陆续上线。这篇博文中介绍的技术成果来自许多 Cursor 员工的共同努力，包括 Lukas Moller、Yash Gaitonde、Wilson Lin、Jason Ma、Devang Jhabakh 和 Jediah Katz。如果你有兴趣用 AI 解决最具挑战性和雄心的编码任务，我们非常期待与你交流。欢迎通过 [hiring@cursor.com](mailto:hiring@cursor.com) 与我们联系。 ■



登录

下载

一个整体结构。当您看到由未输入代码的，它可以从具有可执行力的作者 `dedion`，并可以将它们理解为一个整体单元。文件还支持更强大的搜索能力——Agent 可以使用完整的 `rg` 参数，甚至利用 `jq` 来过滤工具描述。↩

← 上一篇文章  
面向安全和平台团队的 hooks

下一篇文章 →  
使用 Agent 编码的最佳实践

产品

Agent

企业

代码审查

Tab

CLI

云端 Agent

定价

资源

下载

更新日志

文档

论坛

状态

未来

应用市场

公司

职业机会

博客

社区

学生

品牌

Anysphere

法律信息

服务条款

隐私政策

数据使用

安全性

连接

X

LinkedIn

YouTube